download the project, see appendix B, specifically sections B.3 and B.4. Now, you're ready to verify the contract that the web application consumer created from chapter 4.

6.3 Provider contract tests between a web application and data provider

Let's revisit the consumer contract test that our web application from chapter 4 has created:

```
...

const EXPECTED_BODY = { id: 1, name: "My movie", year: 1999 };

describe('Movies Service', () => {
  describe('When a GET request is made to /movies', () => {
    test('it should return all movies', async () => {
      provider
        .uponReceiving('a request to all movies')
        .withRequest({
          method: 'GET',
          path: '/movies',
        })
        .willRespondWith({
          status: 200,
          body: MatchersV3.eachLike(EXPECTED_BODY),
        });

      await provider.executeTest(async mockProvider => {
        const movies = await fetchMovies(mockProvider.url);
        expect(movies[0]).toEqual(EXPECTED_BODY);
      });
    });
  });
});
```

So far, we have one interaction to verify, but further in this chapter, we'll create one more consumer test to demonstrate provider states. In this interaction, the movies API provider is expected to respond with a status code of 200 and a body that is similar to `EXPECTED_BODY`.

6.3.1 Writing the provider test

Let's start writing the provider contract test. This will be broken down into the following code and shown in figure 6.2: